Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 3

«Синхронизация процессов»

по дисциплине «Операционные системы»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Югай П.Э

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3

Ход работы 3-8

1. Теоретическая часть 3-4

2. Практическая часть 4-8

Вывод 9

Листинг «narrow-bridge.c» 10-12

ЦЕЛЬ РАБОТЫ

Цель лабораторной работы – изучение примитивов синхронизации и методов работы с ними, решение классической задачи узкого моста и тестирование решения в рамках операционной системы Pintos.

ХОД РАБОТЫ

1. Теоретическая часть

В этой работе разбирается, как синхронизировать процессы на примере задачи про «узкий мост». Есть мост, и несколько машин (наших процессов) хотят по нему проехать. Основная задача – организовать движение так, чтобы не было аварий и мост не оказался перегружен. По правилам, движение по мосту может идти только в одну сторону, на мосту не должно быть больше двух машин одновременно, а скорая помощь должна проезжать без очереди.

Для решения таких задач часто используют семафоры – это такие простые счетчики с двумя командами: P (или down) – попытаться занять ресурс (если занято – ждем), и V (или up) – освободить ресурс и разбудить кого-то, кто ждет. В Pintos эти команды работают автоматически, поэтому, если правильно использовать семафоры, не нужно отключать прерывания или самому переключать потоки. Это хороший способ синхронизации, который работает предсказуемо и не заставляет процессы ждать впустую.

Как это можно сделать для «узкого моста»? Нам понадобятся переменные или семафоры, чтобы знать, в какую сторону сейчас едут машины, сколько их на мосту и кто ждет своей очереди. Функция narrow_bridge_init() подготовит эти переменные. В функции arrive_bridge() процесс смотрит, что происходит: если направление совпадает и на мосту есть место – он едет; если нет – ждет на специальном семафоре. Функция exit_bridge() уменьшает счетчик машин на мосту, если нужно – меняет направление движения и будит тех, кто ждет (скорую помощь – в первую очередь). Так мы не допустим встречного движения и не перегрузим мост.

Важный момент: синхронизировать нужно через семафоры или замки, нельзя напрямую лезть во внутренние переменные семафоров или вызывать thread_block/thread_unblock. Если правильно организовать очереди и вовремя снимать блокировки, никто не будет ждать слишком долго, если система вообще позволяет проехать.

2. Практическая часть

Задача практической работы — разработать и проверить алгоритм синхронизации для решения проблемы узкого моста в Pintos. Требуется обеспечить безопасный проезд машин в одном направлении, соблюдая лимит на количество одновременно находящихся на мосту автомобилей, а также предоставить приоритетный проезд машинам скорой помощи.

Было реализовано управление доступом к мосту в файле narrow-bridge.c. Изменения позволяют: ограничить число машин на мосту до двух, пропускать транспорт только в одну сторону за раз и давать приоритет машинам скорой помощи. Для этого были добавлены общие переменные, которые отражают состояние моста, массив семафоров для организации очередей по направлениям и приоритетам.

Тип и имя переменной	Назначение / объяснение
`struct lock bridge_lock;`	Замок, к-ый защищает доступ к общим переменным (on_bridge, waiting, direction).
`struct semaphore wait_lock[2][2];`	Семафоры ожидания для каждой комбинации направление/приоритет.
`int waiting[2][2];`	Счётчики ожидающих потоков в очередях для направлений/приоритетов.
`int direction;`	Текущее направление движения (left/right или -1 — не задано).

Также были написаны функции narrow_bridge_init() для инициализации, arrive_bridge() для въезда на мост, exit_bridge() для съезда с него и вспомогательные функции admit() и maybe_flip_direction().

Сигнатура (тип и имя)	Параметры	Что делает
`void narrow_bridge_init(void)`	void	Инициализация lock-ов, семафоров и счётчиков состояния моста.
`void admit(void)`	void	Выдаёт пропуска ожидающим в текущем направлении, учитывая приоритет скорой помощи.
`void maybe_flip_direction(void)`	void	Определяет, нужно ли сменить направление движения (например, если мост пуст и есть ожидающие с другой стороны).
`void arrive_bridge(enum car_priority prio, enum car_direction dir)`	enum car_priority prio, enum car_direction dir	Процедура входа: проверяет возможность въезда на мост, при необходимости блокирует поток в соответствующей

		очереди.
`void exit_bridge(enum car_priority prio UNUSED, enum car_direction dir UNUSED)`	enum car_priority prio UNUSED, enum car_direction dir UNUSED	Процедура выхода: уменьшает on_bridge, при необходимости переключает направление и пробуждает ожидателей.

В моей реализации все счётчики и состояние моста защищены одним замком bridge_lock, чтобы не было гонок при одновременном доступе. Потоки, которые не могут сейчас заехать, ждут на своих семафорах wait_lock[направление][приоритет] — по одному семафору для каждой комбинации направление × приоритет. Функция admit() выдаёт «пропуска» на мост: пока есть свободные места, сначала пускаются машины скорой, потом обычные, всё в текущем направлении. Если на мосту никого нет (on_bridge == 0), вызывается maybe_flip_direction() — проверяет, кого пропустить следующими и при необходимости меняет направление. Такой подход простой, его легко проверить в Pintos и он не использует активного ожидания — потоки реально блокируются (sema_down) и просыпаются только при sema_up.

Ограничение количества машин задаётся константой MAX_VEHICLES_ON_BRIDGE (в задании = 2). Текущее число машин хранится в on_bridge. Допуск новых машин происходит только если on_bridge < MAX_VEHICLES_ON_BRIDGE; при выдаче пропуска on_bridge инкрементируется, при выходе с моста — декрементируется. Поскольку все изменения выполняются под bridge_lock, on_bridge никогда не превысит заданного лимита и не будет рассинхронизации счётчиков.

Для скорых реализованы отдельные очереди (семафоры) и счётчики ожиданий. В функции admit() при раздаче пропусков сначала проверяются и пробуждаются ожидающие скорые в текущем направлении — только если скорых нет или нет свободных мест, пробуждаются обычные машины. Также при on_bridge == 0 вызывается maybe_flip_direction(), которая учитывает наличие скорых с противоположной стороны и может немедленно переключить направление в их пользу. Таким образом скорая получает приоритет: она не ждет, пока пройдут обычные машины, а получает пропуск раньше, что обеспечивает требование «проезжают вне очереди».

Для проверки работы разработанного алгоритма был запущен тест pintos --qemu -- -q run "narrow-bridge 4 3 4 3".

```
Executing 'narrow-bridge 4 3 4 3':
(narrow-bridge) begin
(narrow-bridge) Vehicle:    1, prio: norm, direct: l -> r, ticks=  30
(narrow-bridge) Vehicle:    2, prio: norm, direct: l -> r, ticks=  30
(narrow-bridge) Vehicle:    8, prio: emer, direct: l -> r, ticks=  40
(narrow-bridge) Vehicle:    9, prio: emer, direct: l -> r, ticks=  40
(narrow-bridge) Vehicle:   10, prio: emer, direct: l -> r, ticks=  50
(narrow-bridge) Vehicle:   11, prio: emer, direct: l -> r, ticks=  50
(narrow-bridge) Vehicle:   12, prio: emer, direct: l <- r, ticks=  60
(narrow-bridge) Vehicle:   13, prio: emer, direct: l <- r, ticks=  60
(narrow-bridge) Vehicle:   14, prio: emer, direct: l <- r, ticks=  70
(narrow-bridge) Vehicle:    5, prio: norm, direct: l <- r, ticks=  70
(narrow-bridge) Vehicle:    6, prio: norm, direct: l <- r, ticks=  81
(narrow-bridge) Vehicle:    7, prio: norm, direct: l <- r, ticks=  81
(narrow-bridge) Vehicle:    3, prio: norm, direct: l -> r, ticks=  91
(narrow-bridge) Vehicle:    4, prio: norm, direct: l -> r, ticks=  91
(narrow-bridge) end
Execution of 'narrow-bridge 4 3 4 3' complete.
```

Рис. 1 «Вывод теста “narrow-bridge 4 3 4 3”»

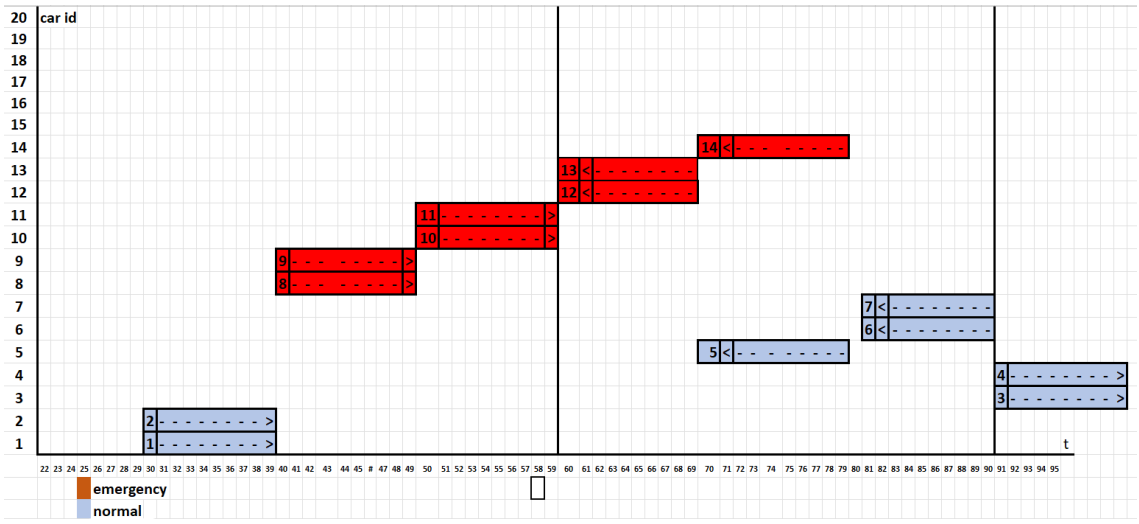


Рис. 2 «Диаграмма к выводу теста “narrow-bridge 4 3 4 3”»

По выводу этого теста (см. рис. 1) построена временная диаграмма пересечений моста(см. рис. 2): по оси X отложены тики времени (ticks из лога), по оси Y — идентификаторы машин. Каждая полоса на диаграмме соответствует интервалу ticks .. ticks+9 (время выполнения cross_bridge() в тесте), красным отмечены автомобили скорой помощи (emergency), синим — обычные (normal). Вертикальная линия на диаграмме показывает момент, после которого началось движение в противоположном направлении.

Диаграмма демонстрирует правильную работу примитивов синхронизации: ограничение числа машин на мосту соблюдается, смена направления происходит при пустом мосту и при наличии скорых и обычных машин с другой стороны моста, а автомобили скорой проходят вне очереди.

ВЫВОД

В ходе работы реализован алгоритм синхронизации для задачи «узкого моста» (файлы: narrow-bridge.c): введены примитивы защиты состояния моста, очереди ожидания и функции narrow_bridge_init(), arrive_bridge() и exit_bridge(). По результатам теста narrow-bridge 4 3 4 3 показано, что одновременно на мосту находится не более двух машин, направление движения сохраняется односторонним, а машины скорой помощи получают приоритет.

Листинг «narrow-bridge.c»

```
/* File for 'narrow_bridge' task implementation.
   SPbSTU, IBKS, 2017 */

#include <stdio.h>
#include <stdbool.h>
#include "tests/threads/tests.h"
#include "threads/thread.h"
#include "threads/synch.h"
#include "narrow-bridge.h"

#define MAX_VEHICLES_ON_BRIDGE 2
struct lock bridge_lock;
struct semaphore wait_lock[2][2];
int waiting[2][2] = {{0,0},{0,0}};
int on_bridge_em[2] = {0,0};
int on_bridge = 0; /* 0...MAX_VEHICLES_ON_BRIDGE */
int direction = -1;
// Called before test. Can initialize some synchronization objects.
void narrow_bridge_init(void)
{
     lock_init(&bridge_lock);

   sema_init(&wait_lock[dir_left][car_normal],0);
   sema_init(&wait_lock[dir_left][car_emergency],0);
   sema_init(&wait_lock[dir_right][car_normal],0);
   sema_init(&wait_lock[dir_right][car_emergency],0);
}

/* Выдача пропусков на свободные места */
void admit(void)
{
   int slots = MAX_VEHICLES_ON_BRIDGE - on_bridge;
   if (slots==0) return; // Мест нет, сразу выходим

   int cur = direction;  // Чья щас очередь
   int other = cur ^ 1; /* 0 <-> 1 - противоположная сторона*/



   /* === ПРИОРИТЕТ ДЛЯ ПАРЫ ОБЫЧНЫХ, НО ТОЛЬКО КОГДА СКОРЫХ НЕТ ===
*/
   if(waiting[cur][car_emergency]==0 &&
      waiting[other][car_emergency]==0 &&
      on_bridge_em[0]==0&&
      on_bridge_em[1]==0)
   {

      /* Мост пуст - если у нас <2 обычных,
         а у них >=2 - отдаём мост им */
      if(on_bridge == 0 &&
         waiting[cur][car_normal]<2 &&
         waiting[other][car_normal]>=2)
      {
         direction=other;
```

```
            return;
        }

        /* На мосту осталось 1 место - не даём
           проскочить одиночному "нормалу", если
           у другой стороны есть пара обычных.
           Ждём опустошения моста */
        if((MAX_VEHICLES_ON_BRIDGE - on_bridge)==1 &&
           waiting[cur][car_normal]>=1 &&
           waiting[other][car_normal]>=2)
        {
           return;
        }

    }

    /* === ПРИОРИТЕТ ПАРЫ СКОРЫХ === */

    /* Мост пуст - если у нас <2 скорых, а у них >= 2 - меняем
направление */
    if(on_bridge == 0 &&
       waiting[cur][car_emergency]<2 &&
       waiting[other][car_emergency]>=2)
    {
       direction=other;
       return;
    }

    /* Осталось 1 место, а у нас есть хотя бы 1 скорая,
       а у другой стороны >= 2 скорых - СТОП. Не даём
       "одной скорой" проскочить, ждём полной очиски
       моста и затем флип на две скорые. */
    if (slots==1 &&
       (waiting[cur][car_emergency] >= 1 || on_bridge_em[cur]==1) &&
       waiting[other][car_emergency]>=2)
    {
       return;
    }


    /* Выпускаем скорых(emergency) текущего направления */
    while (slots>0 && waiting[cur][car_emergency]>0)
    {
       sema_up(&wait_lock[cur][car_emergency]);    //    Выдали    пропуск
скорой
       on_bridge++;
       on_bridge_em[cur]++; // Считаем скорых на мосту по стороне
       waiting[cur][car_emergency]--;
       slots--;
    }

    /* Если на мосту уже едет РОВНО 1 наша скорая, и есть место
       , и у другой стороны 0 скорых - даём ОДНОГО обычного в пару
        */

    if (slots>0 &&
       on_bridge==1 &&
       on_bridge_em[cur] == 1 &&
```

```
      waiting[cur][car_normal]>0 &&
      waiting[other][car_emergency]<=1)
   {
      sema_up(&wait_lock[cur][car_normal]);
      waiting[cur][car_normal]--;
      on_bridge++;
      slots--;


      return;

      //return;  Возможно на другой сторон также 1 скорая может быть
   }

   /* Перед тем как обслуживать обычных, проверяем */
   if (waiting[other][car_emergency]>0)
   {
      return;
   }

   /* Затем обычные текущего направления */
   while (slots>0 && waiting[cur][car_normal] > 0)
   {
      sema_up(&wait_lock[cur][car_normal]);
      on_bridge++;
      waiting[cur][car_normal]--;
      slots--;
   }
}

void maybe_flip_direction(void)
{
   int cur = direction; // Текущее направление обслуживания
   int other = cur ^ 1; // Переключение 0 <-> 1

   // Текущие очереди
   int cur_emergency = waiting[cur][car_emergency];
   int cur_normal = waiting[cur][car_normal];
   int other_emergency = waiting[other][car_emergency];
   int other_normal = waiting[other][car_normal];

   /* Если у другой стороны есть >= 2 скорых
      а у нас <2, переключаемся, чтобы они
      прошли двумя. */
   if (other_emergency >= 2 && cur_emergency < 2)
   {
      direction=other;
      return;
   }

   /* Если есть хоть одна скорая у другой стороны, а у нас нет -
переключаемся */
   if (cur_emergency==0 && other_emergency>0)
   {
      direction = other;
      return;
   }
```

```
   /* Если скорых нет с обеих сторон и у нас
      < 2 обычных, а у другой >=2, переключаемся,
      чтобы пропустить пару обычных. */
   if(cur_emergency == 0 && other_emergency == 0)
   {
      if(cur_normal<2 && other_normal>=2)
      {
         direction=other;
         return;
      }
   }
   /* Если в текущем направлении никого, а в другом кто-то есть -
переключаемся */
   if(cur_emergency+cur_normal==0 && other_emergency+other_normal>0)
   {
      direction = other;
      return;
   }

   /* Иначе остаёмся на текущем направлении */
}

void arrive_bridge(enum car_priority prio, enum car_direction dir)
{
     lock_acquire(&bridge_lock); // чтобы одновременно машины не лезли
на мост все вместе

   /* Отмечаемся, возможно задаём направление на самом старте */
   waiting[dir][prio]++;
   if (direction==-1)
   {
      direction=dir;
   }

   /* Если мост пуст, можно пересмотреть направление(приоритет скорых)
*/
   if (on_bridge == 0)
   {
      maybe_flip_direction();
   }

   /* Пробуем выдавать пропуска на мост на все свободные места */
   admit();

   lock_release(&bridge_lock);

   /* Ждём свой пропуск */
   sema_down(&wait_lock[dir][prio]);

}

void exit_bridge(enum car_priority prio, enum car_direction dir)
{
   lock_acquire(&bridge_lock);

   /* Машина покинула мост */
   on_bridge--;
```

```
    if (prio==car_emergency)
    {
       on_bridge_em[dir]--;
    }

    /* Если мост опустел - можно сменить направление */
    if (on_bridge == 0)
    {
       maybe_flip_direction();
    }

    /* Выдать пропуска дальше в текущем направлении */
    admit();

    lock_release(&bridge_lock);
}
```